

Self-balancing robot

Wiring diagram, pin tables & firmware

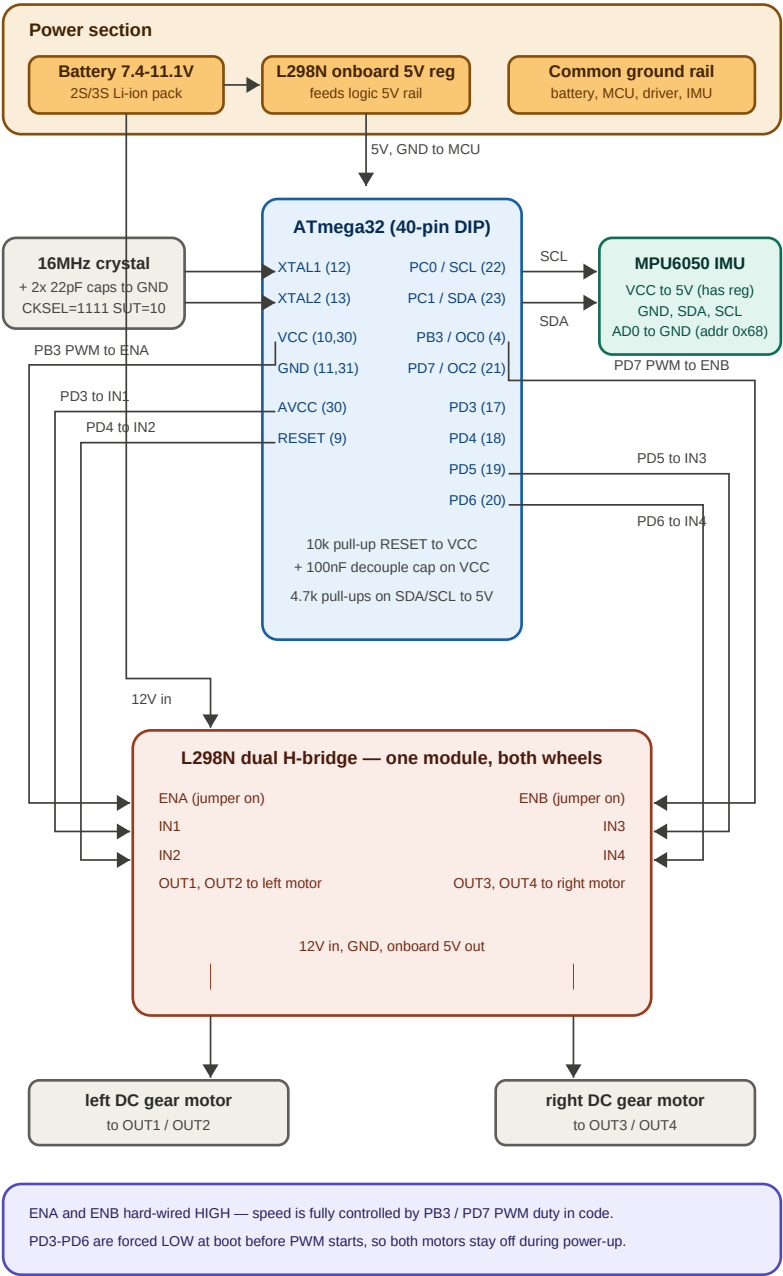
ATmega32 · 16MHz external crystal · MPU6050 · single L298N dual H-bridge

Contents

1. Complete wiring diagram
2. Component-wise pin connection tables
3. Full firmware source code (appendix)

Self-balancing robot — complete wiring diagram

ATmega32 + 16MHz crystal + MPU6050 (I2C) + single L298N dual H-bridge driving both wheels



Component-wise pin connection tables

These tables list every physical connection required to match the firmware exactly. Power and ground connections are listed separately at the end of each table.

1. ATmega32 — core, clock & reset

MCU pin	Connects to	Notes
XTAL1 (pin 12)	Crystal leg 1 + 22pF cap to GND	16MHz external crystal
XTAL2 (pin 13)	Crystal leg 2 + 22pF cap to GND	16MHz external crystal
RESET (pin 9)	10kΩ pull-up to VCC	Active-low reset
VCC (pins 10, 30)	+5V logic rail	From L298N onboard 5V regulator
AVCC (pin 30)	+5V logic rail	Tie to VCC; add ferrite/cap if ADC used
GND (pins 11, 31)	Common ground rail	Shared with driver, IMU, battery

2. ATmega32 ↔ MPU6050 (I2C bus)

MCU pin	Connects to	Notes
PC0 / SCL (pin 22)	MPU6050 SCL	Add 4.7kΩ pull-up to 5V if not on breakout
PC1 / SDA (pin 23)	MPU6050 SDA	Add 4.7kΩ pull-up to 5V if not on breakout
5V rail	MPU6050 VCC	Breakout has onboard regulator
GND rail	MPU6050 GND	Common ground
GND rail	MPU6050 AD0	Sets I2C address 0x68 (code: 0xD0)

3. ATmega32 → L298N — left motor channel (A)

MCU pin	Connects to	Notes
PB3 / OC0 (pin 4)	L298N ENA	Timer0 Fast-PWM speed signal
PD3 (pin 17)	L298N IN1	Direction control bit 1
PD4 (pin 18)	L298N IN2	Direction control bit 2

4. ATmega32 → L298N — right motor channel (B)

MCU pin	Connects to	Notes
PD7 / OC2 (pin 21)	L298N ENB	Timer2 Fast-PWM speed signal
PD5 (pin 19)	L298N IN3	Direction control bit 1
PD6 (pin 20)	L298N IN4	Direction control bit 2

5. L298N — motor outputs & power

MCU pin	Connects to	Notes
OUT1 / OUT2	Left DC gear motor terminals	Polarity sets forward/reverse with IN1/IN2
OUT3 / OUT4	Right DC gear motor terminals	Polarity sets forward/reverse with IN3/IN4
12V (+)	Battery positive (7.4–11.1V pack)	Main motor supply
GND	Battery negative + common ground	Shared with MCU and IMU ground
5V out	ATmega32 VCC/AVCC rail	Only if onboard regulator jumper is fitted

Note: ENA and ENB should be tied permanently HIGH (jumper fitted) since the firmware controls speed entirely through PWM duty cycle on PB3 and PD7, not through separate enable toggling. If your battery exceeds 12V, remove the L298N's onboard 5V regulator jumper and supply the ATmega32 logic rail from a separate 5V regulator, sharing only ground with the motor supply.

Appendix — full firmware source code

Reproduced exactly as supplied. Target: ATmega32 @ 16MHz external crystal, CKSEL = 1111, SUT = 10.

```
/* =====
 * SELF-BALANCING ROBOT - ATmega32 @ 16 MHz EXTERNAL CRYSTAL
 * PID + Complementary Filter + MPU6050 + Dual H-Bridge Motors
 * Corrected version
 * =====
 * Fuse bits required for this code:
 *   External crystal oscillator, 16MHz, startup time max recommended
 *   CKSEL = 1111, SUT = 10 (typical full-swing/low-power xtal, 16K CK + 64ms)
 * ===== */

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <math.h>
#include <stdbool.h>
#include <stdlib.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846f
#endif

#define MIN_ABS_SPEED      110
#define MPU6050_ADDR      0xD0      // 0x68 << 1
#define I2C_TIMEOUT_COUNT 1500

// Set to true if the robot drives forward when it should drive backward
#define INVERT_MOTOR_DIRECTION true // previously false

// ===== PID ENGINE PARAMETERS =====
const float Kp = 35.0f;
const float Kd = 1.4f;
const float Ki = 8.0f;

const float originalSetpoint = 179.40f;
float setpoint = 179.40f; // must be a literal: AVR-GCC requires constant global initializers
float input, output;
float integral = 0.0f, lastInput = 0.0f;

volatile bool loop_trigger = false;
bool isFallen = false;
bool hardwareError = false;

const float motorSpeedFactorLeft = 0.80f;
const float motorSpeedFactorRight = 0.80f;

// ===== FILTER VARIABLES =====
float pitch = 0.0f;
const float dt = 0.01f; // exact 10ms step (Timer1 CTC @ 100Hz)
const float alpha = 0.98f;
int32_t gyro_offset_x = 0;

// ===== OUTPUT CLAMP LIMIT =====
#define OUT_LIMIT 255.0f

// ===== TIMER1 HARDWARE TICK ISR =====
ISR(TIMER1_COMPA_vect) {
    loop_trigger = true;
}

// ===== PERIPHERAL REGISTER SETUP =====
void init_peripherals(void) {
    // Motor PWM pins
    DDRB |= (1 << PB3); // OC0 - Left Motor PWM
    DDRD |= (1 << PD7); // OC2 - Right Motor PWM
    // H-Bridge direction pins
    DDRD |= (1 << PD3) | (1 << PD4) | (1 << PD5) | (1 << PD6);
    // Make sure motors are stopped at boot, before PWM/timers come alive
    PORTD &= ~(1 << PD3) | (1 << PD4) | (1 << PD5) | (1 << PD6);
    // Timer0 (Left PWM) -> Fast PWM, non-inverting, prescaler 8
    TCCR0 = (1 << WGM00) | (1 << WGM01) | (1 << COM01) | (1 << CS01);
    OCR0 = 0;

    // Timer2 (Right PWM) -> Fast PWM, non-inverting, prescaler 8
    TCCR2 = (1 << WGM20) | (1 << WGM21) | (1 << COM21) | (1 << CS21);
    OCR2 = 0;

    // Timer1 -> CTC mode, prescaler 64, fires every 10ms (100Hz)
```

```

TCCR1A = 0;
TCCR1B = (1 << WGM12) | (1 << CS11) | (1 << CS10);
OCR1A = 2499; // (16MHz / (64 * 100Hz)) - 1
TIMSK |= (1 << OCIE1A);

// TWI (I2C) Master, 400kHz Fast Mode @ 16MHz
TWSR = 0x00; // prescaler = 1 (must set explicitly, do not assume POR state)
TWBR = 12; // ((16e6/400e3) - 16) / (2*1) = 12
TWCR = (1 << TWEN); // enable TWI peripheral
}

// Full software+hardware reset of the TWI peripheral after a bus lockup.
void twi_reinit(void) {
    TWCR = 0;
    _delay_us(50);
    TWSR = 0x00;
    TWBR = 12;
    TWCR = (1 << TWEN);
}

// ===== ROBUST I2C DRIVER =====
uint8_t twi_wait(void) {
    uint16_t timeout = I2C_TIMEOUT_COUNT;
    while (!(TWCR & (1 << TWINT))) {
        if (--timeout == 0) {
            hardwareError = true;
            twi_reinit(); // fully restore TWI hardware, not just TWCR=0
            return 0;
        }
    }
    return 1;
}

uint8_t twi_start(void) {
    TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);
    return twi_wait();
}

void twi_stop(void) {
    if (TWCR & (1 << TWEN)) { // only if TWI is actually enabled/alive
        TWCR = (1 << TWINT) | (1 << TWSTO) | (1 << TWEN);
        // Wait for STOP to actually complete (TWSTO clears itself in hardware)
        uint16_t timeout = I2C_TIMEOUT_COUNT;
        while ((TWCR & (1 << TWSTO)) && --timeout) { }
    }
}

uint8_t twi_write(uint8_t data) {
    TWDR = data;
    TWCR = (1 << TWINT) | (1 << TWEN);
    return twi_wait();
}

uint8_t twi_read(uint8_t ack, uint8_t *res) {
    TWCR = (1 << TWINT) | (1 << TWEN) | (ack ? (1 << TWEA) : 0);
    if (!twi_wait()) return 0;
    *res = TWDR;
    return 1;
}

bool mpu_write_reg(uint8_t reg, uint8_t data) {
    if (!twi_start()) return false;
    if (!twi_write(MPU6050_ADDR)) { twi_stop(); return false; }
    if (!twi_write(reg)) { twi_stop(); return false; }
    if (!twi_write(data)) { twi_stop(); return false; }
    twi_stop();
    return true;
}

// ===== MPU6050 CORE =====
void init_mpu(void) {
    _delay_ms(150);

    mpu_write_reg(0x6B, 0x00); // wake MPU6050
    _delay_ms(10);
    mpu_write_reg(0x1A, 0x03); // DLPF ~42Hz
    _delay_ms(10);
    mpu_write_reg(0x1B, 0x00); // gyro full range +-250 deg/s
    _delay_ms(10);

    uint8_t h, l;
    long total_gx = 0;
    int successful_samples = 0;
    for (int i = 0; i < 200; i++) {

```

```

        if (!twi_start()) { twi_stop(); continue; }
        if (!twi_write(MPU6050_ADDR)) { twi_stop(); continue; }
        if (!twi_write(0x43)) { twi_stop(); continue; }

        if (!twi_start()) { twi_stop(); continue; }
        if (!twi_write(MPU6050_ADDR | 0x01)) { twi_stop(); continue; }

        if (!twi_read(1, &h) || !twi_read(0, &l)) { twi_stop(); continue; }
        twi_stop();

        total_gx += (int16_t)((h << 8) | l);
        successful_samples++;
        _delay_ms(2);
    }

    if (successful_samples > 0) {
        gyro_offset_x = total_gx / successful_samples;
    }
}

void update_pitch(void) {
    uint8_t raw_data[14];

    if (!twi_start()) { hardwareError = true; return; }
    if (!twi_write(MPU6050_ADDR)) { twi_stop(); hardwareError = true; return; }
    if (!twi_write(0x3B)) { twi_stop(); hardwareError = true; return; }

    if (!twi_start()) { twi_stop(); hardwareError = true; return; }
    if (!twi_write(MPU6050_ADDR | 0x01)) { twi_stop(); hardwareError = true; return; }

    for (uint8_t i = 0; i < 13; i++) {
        if (!twi_read(1, &raw_data[i])) { twi_stop(); hardwareError = true; return; }
    }
    if (!twi_read(0, &raw_data[13])) { twi_stop(); hardwareError = true; return; }
    twi_stop();

    hardwareError = false;

    int16_t ax = (raw_data[0] << 8) | raw_data[1];
    int16_t ay = (raw_data[2] << 8) | raw_data[3];
    int16_t az = (raw_data[4] << 8) | raw_data[5];
    int16_t gx = (raw_data[8] << 8) | raw_data[9];

    float ax_f = (float)ax;
    float az_f = (float)az;
    float accel_angle = atan2f((float)ay, sqrtf((ax_f * ax_f) + (az_f * az_f))) * 180.0f / (float)M_PI;
    float gyro_rate = (float)(gx - gyro_offset_x) / 131.0f;

    // Complementary filter
    pitch = alpha * (pitch + gyro_rate * dt) + (1.0f - alpha) * accel_angle;
    input = pitch + 180.0f;
}

// ===== PID ENGINE (fixed anti-windup) =====
void compute_pid(void) {
    float error = setpoint - input;

    // Derivative on measurement (eliminates derivative kick)
    float dInput = input - lastInput;

    // Compute candidate output using the CURRENT integral (pre-update)
    float p_term = Kp * error;
    float d_term = -(Kd * dInput / dt);

    // Tentatively add this cycle's integral contribution
    float integral_candidate = integral + (Ki * error * dt);
    float unclamped_output = p_term + integral_candidate + d_term;

    // Conditional integration: only let the integral accumulate further
    // if doing so does NOT push the total output past the saturation limit
    // in the same direction it is already saturated. This is the actual
    // fix for PID windup (clamping integral alone is not sufficient).
    if (unclamped_output > OUT_LIMIT) {
        output = OUT_LIMIT;
        if (error < 0.0f) { // only integrate if it pulls output back down
            integral = integral_candidate;
        }
    } else if (unclamped_output < -OUT_LIMIT) {
        output = -OUT_LIMIT;
        if (error > 0.0f) { // only integrate if it pulls output back up
            integral = integral_candidate;
        }
    } else {
        output = unclamped_output;
    }
}

```

```

    integral = integral_candidate;
}

// Hard safety clamp on the integral term itself (secondary protection)
if (integral > OUT_LIMIT) integral = OUT_LIMIT;
else if (integral < -OUT_LIMIT) integral = -OUT_LIMIT;

lastInput = input;
}

// ===== MOTOR ACTUATION =====
void set_motor_speeds(float speed) {
    if (fabsf(speed) < 1.0f || isFallen || hardwareError) {
        PORTD &= ~(1 << PD3) | (1 << PD4) | (1 << PD5) | (1 << PD6));
        OCR0 = 0;
        OCR2 = 0;
        return;
    }

    float abs_speed = fabsf(speed);
    if (abs_speed > 255.0f) abs_speed = 255.0f;

    uint8_t scaled_pwm = MIN_ABS_SPEED + (uint8_t)((abs_speed / 255.0f) * (255 - MIN_ABS_SPEED));

    uint8_t leftPWM = (uint8_t)(scaled_pwm * motorSpeedFactorLeft);
    uint8_t rightPWM = (uint8_t)(scaled_pwm * motorSpeedFactorRight);

    bool moveForward = (speed > 0);
    if (INVERT_MOTOR_DIRECTION) {
        moveForward = !moveForward;
    }
    if (moveForward) {
        PORTD |= (1 << PD3); PORTD &= ~(1 << PD4);
        PORTD |= (1 << PD5); PORTD &= ~(1 << PD6);
    } else {
        PORTD &= ~(1 << PD3); PORTD |= (1 << PD4);
        PORTD &= ~(1 << PD5); PORTD |= (1 << PD6);
    }

    OCR0 = leftPWM;
    OCR2 = rightPWM;
}

// ===== MAIN LOOP =====
int main(void) {
    init_peripherals();
    init_mpu();

    update_pitch();
    lastInput = input;    // prime state to avoid boot-up derivative kick

    sei();

    while (1) {
        while (!loop_trigger) { }
        loop_trigger = false;

        update_pitch();

        // Fall-detection safety bounds
        if (input < 160.0f || input > 200.0f) {
            isFallen = true;
        }

        if (isFallen || hardwareError) {
            set_motor_speeds(0.0f);
            integral = 0.0f;    // always bleed off integral while down/faulted
            if (!hardwareError && input > 172.0f && input < 182.0f) {
                isFallen = false;
                lastInput = input;    // re-prime so the next dInput isn't a spike
            }
            continue;
        }

        compute_pid();
        set_motor_speeds(output);
    }
}

```